

Program 3: Singly Linked List: Traversal and Display

Aim

To write a C program that traverses a singly linked list and displays its elements, both forward and in reverse order.

Theory

A linked list node holds a **data** field and a **next** pointer; **head** points to the first node and the last node points to **NULL**.

Traversal means visiting every node once, starting at head and following next pointers until **NULL**. A temporary pointer is used so that head is never lost.

Iterative traversal uses a loop; **recursive traversal** calls itself with `node->next`. Both take $O(n)$ time.

A singly linked list can only move forward. To print it in reverse without changing any links, recursion is used: the recursive call is made first and the value is printed while returning.

Algorithm

1. Read `n` and create the list by inserting each value at the end.
2. Iterative: `temp = head`; while `temp != NULL`, print `temp->data` and set `temp = temp->next`.
3. Recursive forward: if node is **NULL** return; print data; recurse on `node->next`.
4. Recursive reverse: if node is **NULL** return; recurse on `node->next`; then print data.

C Source Code

```
/*
 * Program : Singly Linked List - Traversal and Display
 *
 * THEORY:
 * - Traversal means visiting every node exactly once, starting from head and
 *   following the next pointers until NULL is reached.
 * - A temporary pointer is used so that head is never lost.
 * - Iterative traversal uses a loop; recursive traversal calls itself with
 *   node->next. Both take  $O(n)$  time.
 * - A singly linked list can only move FORWARD. To print in reverse order
 *   without changing links, recursion is used (print after the recursive call).
 */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *insertEnd(struct Node *head, int val) {
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->data = val;
    n->next = NULL;
    if (head == NULL) return n;
    struct Node *t = head;
    while (t->next != NULL) t = t->next;
    t->next = n;
    return head;
}

void traverseIterative(struct Node *head) {
    int pos = 1;
    for (struct Node *t = head; t != NULL; t = t->next)
```

```

        printf("Node %d : data = %d\n", pos++, t->data);
    }

void displayChain(struct Node *head) {
    for (struct Node *t = head; t != NULL; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}

void traverseRecursive(struct Node *node) {
    if (node == NULL) return;
    printf("%d ", node->data);
    traverseRecursive(node->next);
}

void traverseReverse(struct Node *node) {
    if (node == NULL) return;
    traverseReverse(node->next);          /* go to the end first */
    printf("%d ", node->data);           /* print while returning */
}

int main() {
    struct Node *head = NULL;
    int n, val;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d values: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertEnd(head, val);
    }

    if (head == NULL) { printf("List is empty.\n"); return 0; }

    printf("\nIterative traversal:\n");
    traverseIterative(head);
    printf("\nList display : "); displayChain(head);
    printf("Recursive traversal (forward) : "); traverseRecursive(head); printf("\n");
    printf("Recursive traversal (reverse) : "); traverseReverse(head);   printf("\n");

    while (head != NULL) { struct Node *t = head; head = head->next; free(t); }
    return 0;
}

```

Sample Input

Input: 5 nodes with values 10 20 30 40 50.

Sample Output (program run with the input above)

```

Enter number of nodes: Enter 5 values:
Iterative traversal:
Node 1 : data = 10
Node 2 : data = 20
Node 3 : data = 30
Node 4 : data = 40
Node 5 : data = 50

List display : 10 -> 20 -> 30 -> 40 -> 50 -> NULL
Recursive traversal (forward) : 10 20 30 40 50
Recursive traversal (reverse) : 50 40 30 20 10

```